

Министерство образования и науки
федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Факультет инфокоммуникационных технологий

Отчет по дисциплине: «**Web-программирование**»
Практическая работа 7-8

Выполнила:

Гриндий Е.А.

Группа:

К3322

Проверила:

Марченко Е.В.

Санкт-Петербург,

2025

Цель: создать простой веб-сайт на Django с тремя страницами:

- Главная страница
- Страница "О разработке"
- Страница обратной связи с формой

Задачи:

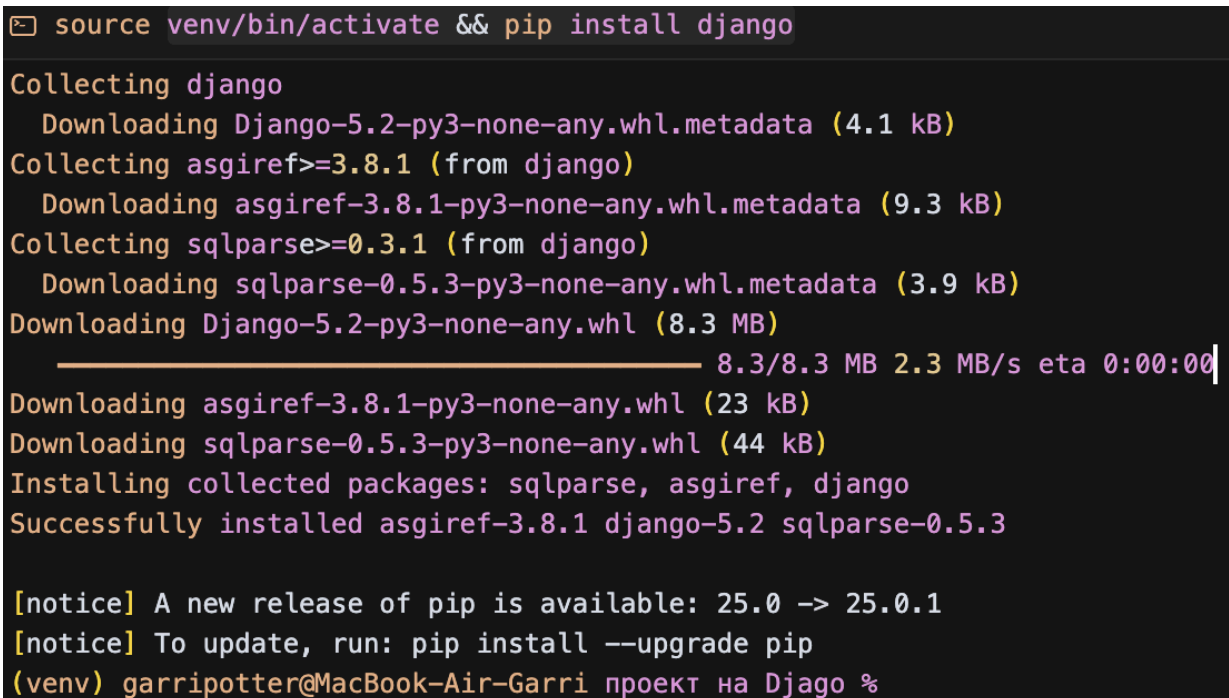
- изучить Django;
- создать базовый Django-проект с тремя страницами;
- для страниц сделать общий дизайн с шаблоном хедера и футера
- реализовать функционал обратной связи с сохранением в базу данных.

Ход работы

Установка Django

Перед началом работы было необходимо установить Django. Это было сделано с помощью команды (рисунок 1):

```
venv/bin/activate && pip install django
```



```
❏ source venv/bin/activate && pip install django
Collecting django
  Downloading Django-5.2-py3-none-any.whl.metadata (4.1 kB)
Collecting asgiref>=3.8.1 (from django)
  Downloading asgiref-3.8.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse>=0.3.1 (from django)
  Downloading sqlparse-0.5.3-py3-none-any.whl.metadata (3.9 kB)
Downloading Django-5.2-py3-none-any.whl (8.3 MB)
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.3/8.3 MB 2.3 MB/s eta 0:00:00
Downloading asgiref-3.8.1-py3-none-any.whl (23 kB)
Downloading sqlparse-0.5.3-py3-none-any.whl (44 kB)
Installing collected packages: sqlparse, asgiref, django
Successfully installed asgiref-3.8.1 django-5.2 sqlparse-0.5.3

[notice] A new release of pip is available: 25.0 -> 25.0.1
[notice] To update, run: pip install --upgrade pip
(venv) garripotter@MacBook-Air-Garri проект на Djago %
```

Рисунок 1 - установка Django в виртуальное окружение

Проверка установленной версии (рисунок 2):

```
(venv) garripotter@MacBook-Air-Garri проект на Django % python3 -m django --version  
5.2
```

Рисунок 2 - проверка

Генерация и регистрация приложения

Инициализация нового Django-проекта:

```
> django-admin startproject myproject .  
  
(venv) garripotter@MacBook-Air-Garri проект на Django %
```

Для реализации функционала было создано отдельное приложение:

```
> python manage.py startapp main  
  
(venv) garripotter@MacBook-Air-Garri проект на Django %
```

По итогу была сгенерирована данная структура проекта с файлами (рисунок 3):

▼ main

> __pycache__

> migrations

> static

> templates

 __init__.py admin.py apps.py forms.py models.py tests.py urls.py views.py

> media

▼ myproject

> __pycache__

 __init__.py asgi.py settings.py urls.py wsgi.py

> static

> venv

≡ db.sqlite3

 manage.py

Рисунок 3 - структура файлов

Django построен на четырех ключевых архитектурных элементах:

1. **Модели (Models)** - отвечают за работу с данными: хранят информацию в БД и предоставляют удобный интерфейс для доступа к ней.
2. **Представления (Views)** - обрабатывают входящие запросы: получают необходимые данные через модели и подготавливают их для отображения.
3. **Шаблоны (Templates)** - определяют визуальное представление данных, полученных от представлений, формируя конечный HTML-код.
4. **Маршрутизатор URL (URL dispatcher)** - анализирует запрашиваемые адреса и направляет их соответствующим обработчикам (представлениям).

Эти компоненты работают согласованно по принципу MVC (Model-View-Controller), где:

- Модели = Model
- Шаблоны = View
- Представления = Controller
- URL-маршрутизатор связывает их все вместе

В файле `myProject/settings.py` добавлено приложение в `INSTALLED_APPS` (рисунок 4):

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'main',  
]
```

Рисунок 4 - добавление приложения в список

Реализация модели

В Django файл **models.py** задаёт структуру БД через Python-классы. Каждая модель становится таблицей, а её атрибуты — колонками. Например, модель `ContactMessage` создаст таблицу для хранения обратной связи, где каждое поле (`name`, `email` и др.) будет отдельным столбцом. Django автоматически генерирует SQL-запросы на основе этих моделей.

```

from django.db import models

class ContactMessage(models.Model):
    name = models.CharField(max_length=100, verbose_name='Имя')
    email = models.EmailField(verbose_name='Email')
    message = models.TextField(verbose_name='Сообщение')
    created_at = models.DateTimeField(auto_now_add=True, verbose_name='Дата создания')

    class Meta:
        verbose_name = 'Сообщение'
        verbose_name_plural = 'Сообщения'
        ordering = ['-created_at']

    def __str__(self):
        return f'{self.name} - {self.email}'

```

Рисунок 5 - модель для обратной связи

Далее модель была зарегистрирована для отображения в админ-панели (рисунок 6).

```

admin.py

from django.contrib import admin
from .models import ContactMessage

@admin.register(ContactMessage)
class ContactMessageAdmin(admin.ModelAdmin):
    list_display = ('name', 'email', 'created_at')
    search_fields = ('name', 'email')
    list_filter = ('created_at',)

```

Рисунок 6 - регистрация модели

После создания модели в **models.py** и её регистрации в **admin.py** требуется сгенерировать и применить миграции.

`makemigrations` — создаёт файлы миграций с SQL-инструкциями на основе изменений в моделях.

`migrate` — применяет эти изменения к базе данных.

```
python manage.py makemigrations
```

```
python manage.py migrate
```

```
python manage.py makemigrations
```

```
(venv) garripotter@MacBook-Air-Garri проект на Django % python manage.py makemig
Migrations for 'main':
  main/migrations/0001_initial.py
    + Create model ContactMessage
(venv) garripotter@MacBook-Air-Garri проект на Django %
```

```
python manage.py migrate
```

```
Applying admin.0003_logentry_add_action_flag_choices... OK
Applying contenttypes.0002_remove_content_type_name... OK
Applying auth.0002_alter_permission_name_max_length... OK
Applying auth.0003_alter_user_email_max_length... OK
Applying auth.0004_alter_user_username_opts... OK
Applying auth.0005_alter_user_last_login_null... OK
Applying auth.0006_require_contenttypes_0002... OK
Applying auth.0007_alter_validators_add_error_messages... OK
Applying auth.0008_alter_user_username_max_length... OK
Applying auth.0009_alter_user_last_name_max_length... OK
Applying auth.0010_alter_group_name_max_length... OK
Applying auth.0011_update_proxy_permissions... OK
Applying auth.0012_alter_user_first_name_max_length... OK
Applying main.0001_initial... OK
Applying sessions.0001_initial... OK
(venv) garripotter@MacBook-Air-Garri проект на Django %
```

Рисунок 7 - создание базы данных

Реализация формы

В Django формы обеспечивают безопасный сбор пользовательских данных. В проекте используется **ModelForm** - удобный инструмент, который автоматически генерирует форму на основе существующей модели (Рисунок 8).

```
forms.py
from django import forms
from .models import ContactMessage

class ContactForm(forms.ModelForm):
    class Meta:
        model = ContactMessage
        fields = ['name', 'email', 'message']
        widgets = {
            'name': forms.TextInput(attrs={'class': 'form-control', 'placeholder': 'Введите ваше имя'}),
            'email': forms.EmailInput(attrs={'class': 'form-control', 'placeholder': 'Введите ваш email'}),
            'message': forms.Textarea(attrs={'class': 'form-control', 'placeholder': 'Введите ваше сообщение'}),
        }
        labels = {
            'name': 'Ваше имя',
            'email': 'Ваш email',
            'message': 'Ваше сообщение',
        }
```

Рисунок 8 - создание формы

- **ModelForm** — автоматически генерирует форму с полями, соответствующими модели.
- **Meta** — внутренний класс для настроек:
 - `model = ContactMessage` — указывает, какая модель используется.
 - `fields` — список полей модели, которые должны быть в форме.
 - `widgets` — определяет HTML-элементы и их атрибуты
 - `labels` — определяет названия полей

Django автоматически проверяет:

1. Обязательные поля (если в модели поле `blank=False`).
2. Типы данных:

- o CharField → текст.
 - o EmailField → валидный email-адрес.
3. Максимальную длину (max_length).

Добавим обработку формы в файл `main/views.py` (рисунок 9).

Файл `views.py` — это один из ключевых файлов в Django-приложении, который отвечает за логику обработки запросов и генерацию ответов (HTML-страниц, JSON и т.д.).

View (представление) — это функция или класс, которая:

1. Принимает HTTP-запрос (request) от пользователя.
2. Обработывает данные (из БД, форм и т.д.).
3. Возвращает HTTP-ответ (например, HTML-страницу или редирект).

```
views.py
from django.shortcuts import render, redirect
from django.views.generic import TemplateView
from django.views.generic.edit import FormView
from .forms import ContactForm
from .models import ContactMessage

class HomeView(TemplateView):
    template_name = 'main/home.html'

class AboutView(TemplateView):
    template_name = 'main/about.html'

class ContactView(FormView):
    template_name = 'main/contact.html'
    form_class = ContactForm
    success_url = '/about/'

    def form_valid(self, form):
        form.save()
        return super().form_valid(form)

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context['form'] = self.form_class()
        return context
```

Рисунок 9 - view-файл

Логика работы contact:

Если запрос GET:

- Создается пустой экземпляр формы
- Передается в шаблон
- Пользователь видит пустую форму для заполнения

Если запрос POST :

- Django автоматически проверяет данные
- Если данные валидны, вызывается form_valid
- Данные сохраняются в базу
- Пользователь перенаправляется на /about/

Настройка маршрутизации

Напишем основные URL-маршруты (рисунок 10) в главном файле маршрутизации myProject/urls.py:

```
myproject > 📄 urls.py
1  """
2  URL configuration for myproject project.
3
4  The `urlpatterns` list routes URLs to views. For more information please see:
5  | https://docs.djangoproject.com/en/5.2/topics/http/urls/
6  | Examples:
7  | Function views
8  |     1. Add an import: from my_app import views
9  |     2. Add a URL to urlpatterns: path('', views.home, name='home')
10 | Class-based views
11 |     1. Add an import: from other_app.views import Home
12 |     2. Add a URL to urlpatterns: path('', Home.as_view(), name='home')
13 | Including another URLconf
14 |     1. Import the include() function: from django.urls import include, path
15 |     2. Add a URL to urlpatterns: path('blog/', include('blog.urls'))
16 | """
17 from django.contrib import admin
18 from django.urls import path, include
19 from django.conf import settings
20 from django.conf.urls.static import static
21
22 urlpatterns = [
23     path('admin/', admin.site.urls),
24     path('', include('main.urls')),
25 ]
26
27 if settings.DEBUG:
28     urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
29     urlpatterns += static(settings.STATIC_URL, document_root=settings.STATIC_ROOT)
30
```

Рисунок 10 - основные маршруты

URL-маршруты (рисунок 11) приложения в файле `main/urls.py`:

```
main >  urls.py
1  from django.urls import path
2  from .views import HomeView, AboutView, ContactView
3
4  urlpatterns = [
5      path('', HomeView.as_view(), name='home'),
6      path('about/', AboutView.as_view(), name='about'),
7      path('contact/', ContactView.as_view(), name='contact'),
8  ]
```

Рисунок 11 - URL-маршруты

Шаблоны и фронтенд

Для реализации отображения контента был создан файл `main/templates/main/base.html` (рисунок 12-13):

```

main > templates > main > > base.html > html > body > main
1  <!DOCTYPE html>
2  <html lang="ru">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>{% block title %}Мой сайт{% endblock %}</title>
7      {% load static %}
8      <link rel="stylesheet" href="{% static 'main/css/style.css' %}">
9  </head>
10 <body>
11     <header>
12         <div class="header-container">
13             <a href="{% url 'home' %}" class="logo">Мой сайт</a>
14             <nav>
15                 <ul class="nav-menu">
16                     <li class="nav-item">
17                         <a href="{% url 'home' %}" class="nav-link">Главная</a>
18                     </li>
19                     <li class="nav-item">
20                         <a href="{% url 'about' %}" class="nav-link">О разработке</a>
21                     </li>
22                     <li class="nav-item">
23                         <a href="{% url 'contact' %}" class="nav-link">Обратная связь</a>
24                     </li>
25                 </ul>
26             </nav>
27         </div>
28     </header>
29
30     <main>
31         {% block content %}
32         {% endblock %}
33     </main>
34
35     <footer>
36         <div class="header-container">
37             <p>© 2024 Мой сайт. Все права защищены.</p>
38         </div>
39     </footer>
40 </body>
41 </html>

```

Рисунок 12 - основной шаблон

Добавим дочерние шаблоны (рисунок 13):

```

main > templates > main > <> contact.html > div.contact-form > form
1  {% extends 'main/base.html' %}
2
3  {% block title %}Обратная связь{% endblock %}
4
5  {% block content %}
6  <div class="contact-form">
7      <h1 class="page-title">Обратная связь</h1>
8      <form method="post">
9          {% csrf_token %}
10         ... {% for field in form %}
11             <div class="form-group">
12                 <label class="form-label" for="{{ field.id_for_label }}">{{ field.label }}</label>
13                 {{ field }}
14                 {% if field.errors %}
15                     <div class="alert alert-danger">
16                         {{ field.errors }}
17                     </div>
18                 {% endif %}
19             </div>
20         {% endfor %}
21         <button type="submit" class="btn">Отправить</button>
22     </form>
23 </div>
24 {% endblock %}

```

Рисунок 14 - дочерний шаблон обратной связи

Создание суперпользователя

Создадим суперпользователя для проверки отправки данных формы (рисунок 15 - 17).

```

garripotter@MacBook-Air-Garri проект на Django % source venv/bin/activate && python manage.py createsuperuser
source venv/bin/activate && python manage.py createsuperuser
Username (leave blank to use 'garripotter'): superuser
Email address: SuperUser@mail.ru
Password:
Password (again):
This password is too short. It must contain at least 8 characters.
This password is entirely numeric.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
(venv) garripotter@MacBook-Air-Garri проект на Django %

```

Рисунок 15 - создание суперпользователя

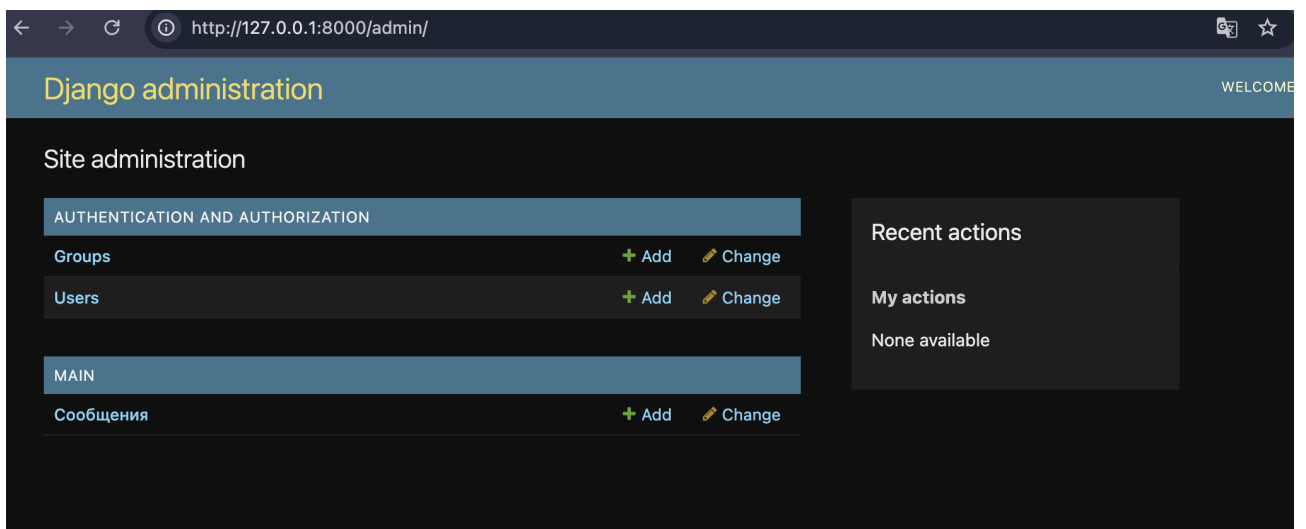


Рисунок 17 – проверка

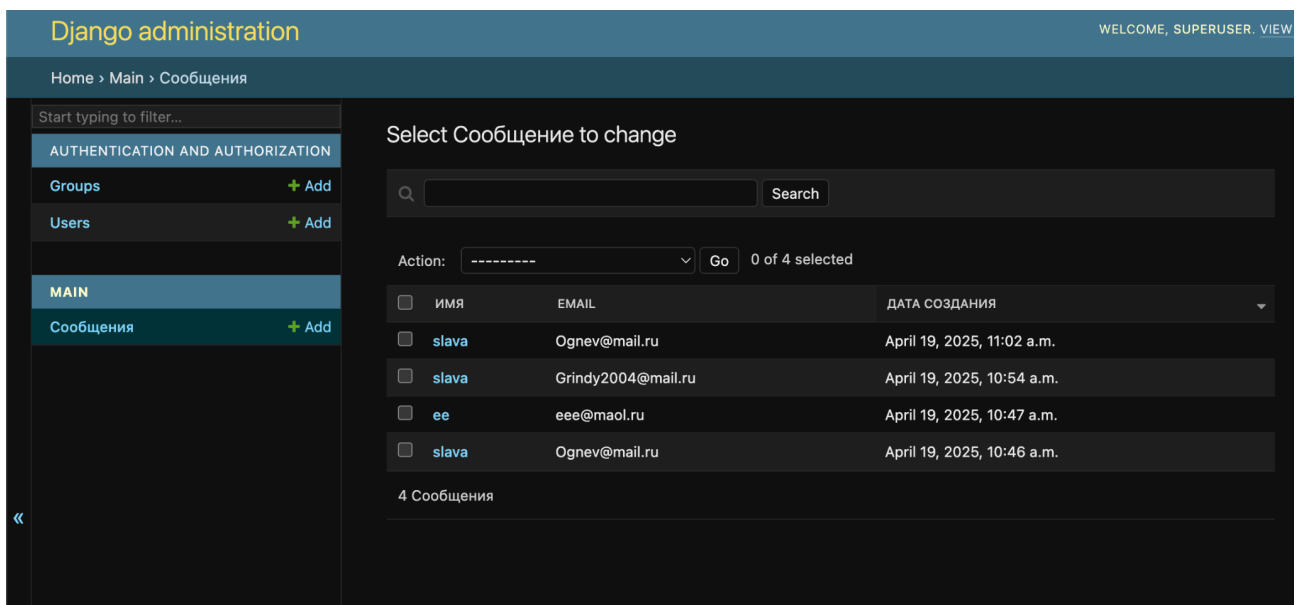


Рисунок 18 — Данные в базе отправленные с формы

Проверка работы приложения

```
(venv) garripotter@MacBook-Air-Garri проект на Djago %  
python3 manage.py runserver
```

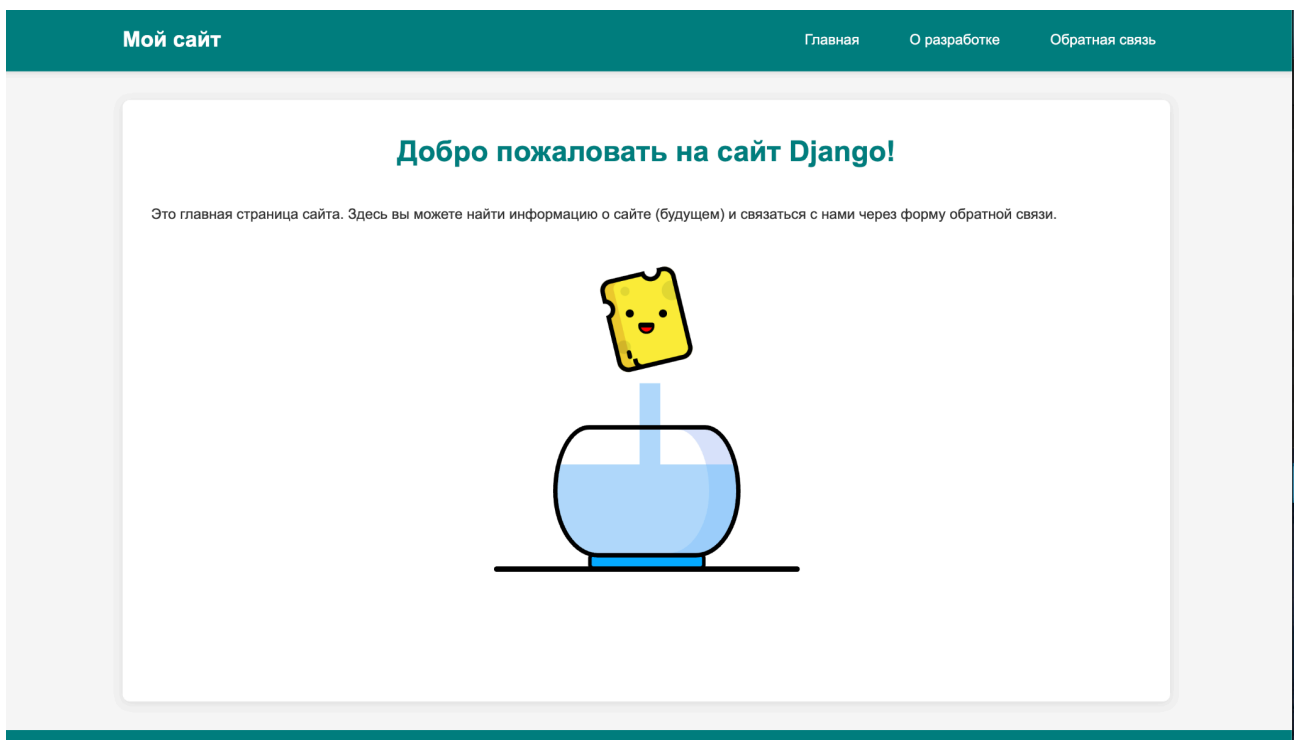


Рисунок 18 - итог

Вывод: в ходе лабораторной работы были изучены основы фреймворка Django. Создано Django-приложение с тремя страницами (главная, "О разработке", обратная связь), а так же реализовано сохранение данные в базу данных.